

Projektdokumentation – XRechnung Validator

1. Überblick

Der **XRechnung Validator** ist eine Webanwendung zur Prüfung elektronischer Rechnungen.

Ein Benutzer lädt eine XML-Rechnung hoch; die Anwendung prüft sie mit dem

Mustangproject-Validator (Version 2.24.0) und zeigt an, ob die Rechnung **gültig** oder **ungültig** ist – zusammen mit dem vollständigen Prüfbericht.

Unterstützt werden die vom Mustangproject abgedeckten Formate, insbesondere **XRechnung** sowie **ZUGFeRD / Factur-X**.

Die Anwendung besteht aus zwei Teilen:

- **Backend** – eine REST-API (Spring Boot, Java), die die eigentliche Validierung durchführt.
- **Frontend** – eine Weboberfläche (Vue 3), über die Dateien hochgeladen und Ergebnisse angezeigt werden.

2. Funktionsweise (Ablauf)

1. Der Benutzer wählt im Frontend eine XML-Datei aus oder legt sie per Drag & Drop ab.

2. Mit Klick auf **Validieren** wird die Datei an das Backend gesendet.
3. Das Backend übergibt die Datei an den Mustang-Validator und ermittelt das Ergebnis.
4. Das Backend antwortet mit einem Status (`VALID` / `INVALID`) und dem vollständigen Prüfbericht als XML.
5. Das Frontend stellt das Ergebnis übersichtlich dar: Status, Kenndaten (Profil, geprüfte Regeln, Dauer) sowie eventuelle Fehler und Hinweise.

3. Architektur



Frontend und Backend sind getrennte Anwendungen und kommunizieren ausschließlich über die REST-Schnittstelle.

4. Verwendete Technologien

Bereich	Technologie
Backend	Java 21, Spring Boot 3.5, Maven
Validator	Mustangproject validator 2.24.0
Frontend	Vue 3, Vite, Axios

5. Projektstruktur

```

xrechnung-validator/
├─ backend/
│   ├─ pom.xml
│   └─ src/main/
│       ├─ java/com/example/demo1/
│       │   └─ Demo1Application.java      #
│       └─ Startklasse
│           └─ XmlUploadController.java    # REST-
│               └─ Endpunkt
│                   └─ ValidationService.java    #
│                       └─ Validierungslogik (Mustang)
│                           └─ ValidationResponse.java    # Antwort-
│                               └─ Objekt (JSON)
│                                   └─ WebConfig.java      # CORS-
│                                       └─ Konfiguration
│                                           └─ resources/
│                                               └─ application.properties    # Port,
Upload-Grenzen
├─ frontend/
│   ├─ package.json
│   ├─ index.html
│   └─ src/
│       ├─ main.js
│       ├─ App.vue
│       └─ components/
│           └─ XRechnungValidator.vue    # gesamte

```

```
Oberfläche + Logik
└─ test-data/                               #
Beispieldateien zum Testen
```

6. Backend im Detail

6.1 Endpunkt

```
POST /upload-xml
Content-Type: multipart/form-data
Formularfeld: "file"
```

6.2 Ablauf im Controller (`XmlUploadController`)

- Prüft, ob eine Datei vorhanden und nicht leer ist.
- Prüft, ob der Dateiname auf `.xml` endet (einfache Vorabprüfung).
- Übergibt die Datei an den `ValidationService`.
- Gibt das Ergebnis als JSON (`ValidationResponse`) zurück.
- Bei Fehlern wird ein Status `ERROR` mit einer Meldung zurückgegeben.

6.3 Validierung (`ValidationService`)

- Für jede Anfrage wird eine neue Instanz von `ZUGFeRDValidator` erzeugt (der Validator ist zustandsbehaftet und darf nicht als gemeinsame Instanz verwendet werden).
- Die Datei wird mit `validate(...)` geprüft.
- Der Gültigkeitsstatus wird über `wasCompletelyValid()` ermittelt – also über das Ergebnis

des Validators selbst, nicht durch Auswerten von Textbausteinen des Berichts. Dadurch bleibt die Logik robust, auch wenn sich das Berichtsformat ändert.

- Zurückgegeben werden Dateiname, Status (`VALID` / `INVALID`) und der vollständige Bericht.

6.4 CORS (`WebConfig`)

Das Frontend läuft während der Entwicklung auf einem anderen Port als das Backend. Damit der Browser die API aufrufen darf, ist CORS für `http://localhost:5173` (die Frontend-Adresse) freigegeben.

7. Frontend im Detail

Die gesamte Oberfläche und Logik befindet sich in der Komponente `XRechnungValidator.vue`.

Funktionen:

- Dateiauswahl über Button oder **Drag & Drop** (mit visueller Rückmeldung).
- Client-seitige Prüfung, ob eine `.xml` -Datei gewählt wurde.
- Anzeige der ausgewählten Datei (Name und Größe).
- **Ladeanzeige** während der Validierung.
- Ergebnisdarstellung:
 - Statusbanner **Gültig** (grün) bzw. **Ungültig** (rot),
 - Kenndaten: Profil, Anzahl geprüfter/fehlgeschlagener Regeln, Dauer, Validator-Version,
 - Listen für Fehler und Hinweise,

- der vollständige XML-Bericht ist zusätzlich einklappbar verfügbar.

- **Zurücksetzen**-Funktion.

Das Frontend liest den XML-Bericht des Backends aus und bereitet ihn benutzerfreundlich auf.

Die Gültigkeitsaussage selbst stammt aus dem Status-Feld des Backends.

8. API-Schnittstelle

Anfrage

```
POST http://localhost:8080/upload-xml
Content-Type: multipart/form-data
file = <XML-Datei>
```

Antwort (JSON)

```
{
  "fileName": "rechnung.xml",
  "status": "VALID",
  "validationResult": "<validation> ...
</validation>",
  "error": null
}
```

Statuswerte

Status	Bedeutung
VALID	Die Rechnung ist konform.
INVALID	Die Rechnung wurde geprüft, ist aber nicht konform.
ERROR	Die Anfrage konnte nicht verarbeitet werden (z. B. leere Datei).

Hinweis: Eine nicht konforme Rechnung führt zu `INVALID` mit HTTP-Status 200 – das ist ein gültiges Prüfergebnis und kein technischer Fehler. `ERROR` wird nur bei tatsächlichen Verarbeitungsproblemen verwendet.

9. Anwendung starten (lokal)

Voraussetzungen: Java 21, Maven, Node.js 20+ und npm.

Backend

```
cd backend
mvn spring-boot:run
```

Das Backend läuft anschließend unter **`http://localhost:8080`**.

Frontend

In einem zweiten Terminal:

```
cd frontend
npm install
```

```
npm run dev
```

Das Frontend ist danach unter **http://localhost:5173** erreichbar.

Beim ersten Start werden Abhängigkeiten aus dem Internet geladen (Maven Central / npm), daher ist beim ersten Mal ein Internetzugang erforderlich.

10. Testen

1. Frontend öffnen: <http://localhost:5173>
2. XML-Datei auswählen oder ablegen und **Validieren** klicken.
3. Zum schnellen Test liegt unter `test-data/not-an-invoice.xml` eine bewusst **ungültige** Datei bei (erwartetes Ergebnis: **Ungültig**).
4. Gültige Beispielrechnungen liefert die offizielle KoSIT-Testsuite:
<https://github.com/itplr-kosit/xrechnung-testsuite>
Ordner `src/test/business-cases/standard/` (erwartetes Ergebnis: **Gültig**).

11. Konfiguration

- **Ports:** Frontend 5173, Backend 8080.
- Ändert sich der **Backend-Port**, muss die Konstante `API_URL` in `frontend/src/components/XRechnungValidator.vue` angepasst werden.
- Ändert sich der **Frontend-Port**, muss der CORS-Ursprung in

`backend/src/main/java/com/example/demo1/WebConfig.java`
angepasst werden.

- **Upload-Grenze:** In `application.properties` auf 20 MB gesetzt (Standard von Spring wäre 1 MB).

12. Hinweise und Grenzen

- Das Backend verwendet Mustang 2.24.0 mit einer bestimmten Regel-Version. Sehr aktuelle Testdateien können daher vereinzelt zusätzliche Hinweise erzeugen oder abweichen. Das ist eine Frage der Versionsstände, kein Fehler der Anwendung.
- Die Dateiprüfung auf `.xml` im Backend ist nur eine einfache Vorabprüfung anhand des Dateinamens; die eigentliche inhaltliche Prüfung übernimmt der Mustang-Validator.
- Die Konfiguration ist für die lokale Nutzung/Entwicklung ausgelegt (CORS nur für localhost).